

LLP TCA2 — Past Paper Step-by-Step Guide

Based on: LLP Class 2025 TCA2 — Advanced Concepts in Low Level Programming

Exam Setup Checklist (Do This First!)

1. Log in using `vleuser` account
2. Download the exam Word document from VLE
3. Save it as `NameSurnameClass` (e.g., `JohnBorgClass1`)
4. Open Visual Studio (NOT VS Code — VS is the full IDE)
5. Create a new solution named `NameSurnameClass`
6. Add projects to the solution as required (Question2, Question3, Question4)
7. At the end: zip everything → upload to VLE

Question 1 — GCC Compilation (AA1.3) [7 marks]

What this question asks:

Use command-line GCC to compile `Greeting.c` through multiple stages and take screenshots.

Step-by-Step:

Before starting:

1. Download `Greeting.c` from VLE
2. Place it in a simple folder, e.g. `C:\exam\` (short path avoids typing errors)
3. Open Command Prompt: press `Win + R`, type `cmd`, press `Enter`

Q1a — Convert to pre-processed code (`.i` file) [2 marks]:

```
gcc -E Greeting.c -o Greeting.i
```

- Type this exactly in the command prompt (make sure you're in the right folder — use `cd C:\exam`)
- Take a screenshot showing the command and the prompt
- Save screenshot as `Q1a`

Q1b — Convert to assembly (`.s` file) [2 marks]:

```
gcc -S Greeting.c -o Greeting.s
```

- Take a screenshot
- Save as Q1b

Q1c — Compile to executable (.exe file) [2 marks]:

```
gcc Greeting.c -o Greeting.exe
```

- Take a screenshot
- Save as Q1c

Q1d — Run the executable and type your firstname [1 mark]:

```
Greeting.exe
```

- When prompted "Enter your name:", type your first name
- The output should show: Good morning, [YourName]
- Take a screenshot showing the output with your name clearly visible
- Save as Q1d

Key things to remember:

- Navigate to the folder first: `cd C:\exam`
- Check GCC is installed: `gcc --version`
- The `-E` flag = pre-process only, `-S` flag = compile to assembly, no flag = full compile
- `-o` specifies the output filename

Question 2 — Functions & Debugging (AA4.1 + KU1.4) [11 marks]

What this question asks:

Write `isNumeric` function, debug it in Visual Studio, then explain `toNumeric`.

Step-by-Step:

Setup:

1. In Visual Studio, create a new project called `Question2`
2. Create a `.c` file (e.g., `question2.c`)

Q2a — Write and test `isNumeric` function [4 marks]:

```

#include <stdio.h>
#include <string.h>
#include <ctype.h>

_Bool isNumeric(char *str) {
    if (!str || strlen(str) == 0) return 0;
    for (int i = 0; str[i] != '\0'; i++) {
        if (!isdigit(str[i])) return 0;
    }
    return 1;
}

int main(void) {
    char test1[] = "12345";
    char test2[] = "123a5";
    char test3[] = "";

    printf("'s' is numeric: %d\n", test1, isNumeric(test1)); // should be 1
    printf("'s' is numeric: %d\n", test2, isNumeric(test2)); // should be 0
    printf("'s' is numeric: %d\n", test3, isNumeric(test3)); // should be 0

    return 0;
}

```

What the function does:

- Checks if the pointer is not NULL and string is not empty
- Loops through every character using `isdigit()` from `<ctype.h>`
- Returns 0 (false) as soon as it finds a non-digit
- Returns 1 (true) if all characters pass

Copy and paste this complete code into the space provided on the exam paper.

Q2b — Debug in Visual Studio [5 marks]:

1. Set a breakpoint on the line `_Bool result = isNumeric(test1);` (Or the first `printf` line):
 - Click in the grey gutter to the left of the line number → a red dot appears
2. Start debugging: Press F5 or click Debug → Start Debugging
3. Add variables to Watch:
 - Go to Debug → Windows → Watch → Watch 1
 - Click in the Name column and type: `test1` → press Enter
 - Also add: `test2`, `str`, `i`
4. Step through the code:
 - Press F10 (Step Over) to move line by line in main
 - When you reach the `isNumeric` call, press F11 (Step Into) to enter the function
 - Watch the value of `str`, the loop variable `i`, and each character being checked

5. Take screenshots showing:

- The breakpoint (red dot) on a line
- The Watch panel showing variable values (e.g., `str = "12345"`, `i = 0`)
- The yellow arrow (current execution point) on a specific line

Save the screenshots as Q2b.

Q2c — Explain how data is passed in/out of `toNumeric` [3 marks]:

Write this explanation:

The function `toNumeric` receives data in through two parameters:

1. `char *s` — a pointer to the string in main. The address of the first character of `str` is passed, so the function can read the string's contents without making a copy.
2. `int *number` — a pointer to the `num` variable in main. The address of `num` is passed using `&num` when calling `toNumeric(str, &num)`.

Data comes out through two channels:

1. The return value `_Bool` — returns 1 (success) or 0 (failure), communicating whether the conversion worked.
2. The pointer parameter `int *number` — inside the function, `*number = atoi(s)` dereferences the pointer and writes the converted integer directly into main's `num` variable. This is how the converted number is passed back out without using the return value.

Question 3 — Dynamic List of Static Structs + Binary File (SE3.3 + SE4.2) [15 marks]

What this question asks:

Create a `saveToFile.c` program that stores dog records in a dynamic list and saves to binary file.

Step-by-Step:

Setup:

1. Create new project `Question3` in the same VS solution
2. Create file `saveToFile.c`
3. Add `#include <stdio.h>`, `#include <stdlib.h>`, `#include <string.h>` at the top

Q3a — Define the struct [1 mark]:

```
struct dog {
    char chipId[15];    // 15-digit chip ID
    char name[30];      // up to 30 chars
```

```
    char breed[30];    // up to 30 chars
};
```

Remember: Use `char name[30]` NOT `char *name` — arrays embed data in the struct which is required for binary file writing.

Q3b — Write `fillRecord` function [1 mark]:

```
struct dog fillRecord(void) {
    struct dog d;
    printf("Enter chip ID (max 14 digits): ");
    scanf("%14s", d.chipId);
    printf("Enter name: ");
    scanf("%29s", d.name);
    printf("Enter breed: ");
    scanf("%29s", d.breed);
    return d;
}
```

Test it in main by calling `fillRecord()` and passing result to `displayRecord`.

Q3c — Write `displayRecord` function [1 mark]:

```
void displayRecord(struct dog d) {
    printf("Chip ID: %s\n", d.chipId);
    printf("Name:    %s\n", d.name);
    printf("Breed:   %s\n", d.breed);
    printf("---\n");
}
```

Q3d — Declare pointer + allocate memory for 3 records [1 mark]:

```
#include <stdlib.h>    // MUST include this for malloc

int main(void) {
    struct dog *dogList = (struct dog*)malloc(3 * sizeof(struct dog));
    if (!dogList) {
        printf("Memory allocation failed!\n");
        return -1;
    }
    // ... rest of code
    free(dogList);
    dogList = NULL;
    return 0;
}
```

Q3e — Display your name + fill 3 records [2 marks]:

```
printf("Student: Your Name Here\n");    // Replace with your actual name
```

```

for (int i = 0; i < 3; i++) {
    printf("\nEnter details for dog %d:\n", i + 1);
    dogList[i] = fillRecord();
}

```

Q3f — Display the 3 records using displayRecord [2 marks]:

```

printf("\n--- Dog Records ---\n");
for (int i = 0; i < 3; i++) {
    displayRecord(dogList[i]);
}

```

Q3g — Save to binary file dogData.bin [2 marks]:

```

printf("\nSaving data to file...\n");

FILE *fp = fopen("dogData.bin", "wb");
if (!fp) {
    printf("Error opening file!\n");
    free(dogList);
    return -1;
}

for (int i = 0; i < 3; i++) {
    fwrite(&dogList[i], sizeof(dogList[i]), 1, fp);
}

fclose(fp);
printf("Data saved to dogData.bin\n");

```

Q3h — Advantage/Disadvantage comment [5 marks]:

Add this comment in your program:

```

/*
 * ADVANTAGE of dynamic list of static structs over array of structs:
 * The size of the list is decided at runtime using malloc(), so memory
 * is only allocated for the exact number of records needed. An array
 * of structs requires the size to be fixed at compile time, which may
 * waste memory if too large or be insufficient if too small. The dynamic
 * list can also be resized at runtime using realloc() if more space is needed.
 *
 * DISADVANTAGE of dynamic list of static structs over array of structs:
 * The programmer is responsible for manually managing memory (malloc and free).
 * This increases code complexity and risks memory leaks if free() is forgotten.
 * An array of structs is simpler – memory is managed automatically by the compiler
 */

```

Complete Question 3 Program:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct dog {
    char chipId[15];
    char name[30];
    char breed[30];
};

struct dog fillRecord(void) {
    struct dog d;
    printf("Enter chip ID (max 14 digits): ");
    scanf("%14s", d.chipId);
    printf("Enter name: ");
    scanf("%29s", d.name);
    printf("Enter breed: ");
    scanf("%29s", d.breed);
    return d;
}

void displayRecord(struct dog d) {
    printf("Chip ID: %s\n", d.chipId);
    printf("Name: %s\n", d.name);
    printf("Breed: %s\n", d.breed);
    printf("---\n");
}

int main(void) {
    printf("Student: Your Name\n");

    struct dog *dogList = (struct dog*)malloc(3 * sizeof(struct dog));
    if (!dogList) {
        printf("Memory allocation failed!\n");
        return -1;
    }

    for (int i = 0; i < 3; i++) {
        printf("\nEnter details for dog %d:\n", i + 1);
        dogList[i] = fillRecord();
    }

    printf("\n--- Dog Records ---\n");
    for (int i = 0; i < 3; i++) {
        displayRecord(dogList[i]);
    }

    printf("\nSaving data to file...\n");
    FILE *fp = fopen("dogData.bin", "wb");
    if (!fp) {
        printf("Error opening file!\n");
        free(dogList);
        return -1;
    }
    for (int i = 0; i < 3; i++) {
        fwrite(&dogList[i], sizeof(dogList[i]), 1, fp);
    }
}

```

```

    }
    fclose(fp);
    printf("Data saved to dogData.bin\n");

    /*
     * ADVANTAGE: Dynamic list size decided at runtime; memory only used as needed.
     * DISADVANTAGE: Manual memory management required; risk of memory leaks.
     */

    free(dogList);
    dogList = NULL;
    return 0;
}

```

Question 4 — Dynamic List of Dynamic Structs + Header File (SE4.3 + SE4.2) [15 marks]

What this question asks:

Create a `readFromFile.c` program that reads records from `dogData.bin` into a dynamic list of dynamic structs, using a header file.

Step-by-Step:

Setup:

1. Create new project `Question4` in the same VS solution
2. Create `readFromFile.c` and `myHeader.h` in this project

Q4a — Create the header file `myHeader.h` [1 mark]:

```

// myHeader.h
#ifndef MYHEADER_H
#define MYHEADER_H

#include <stdio.h>

struct dog {
    char chipId[15];
    char name[30];
    char breed[30];
};

struct dog fillRecord(void);
void displayRecord(struct dog d);

#endif

```

Then in `readFromFile.c`, add at the top:


```
#include "myHeader.h"    // use double quotes for your own file
```

Important: Also copy `fillRecord` and `displayRecord` function bodies into `readFromFile.c` (since this is a single-project, the functions must be defined somewhere).

Q4b — Declare pointer-to-pointer + allocate + set to NULL [1 mark]:

```
int capacity = 10;
struct dog **dList = (struct dog**)malloc(capacity * sizeof(struct dog*));
if (!dList) {
    printf("Allocation failed!\n");
    return -1;
}

// Set ALL pointers to NULL
for (int i = 0; i < capacity; i++) {
    dList[i] = NULL;
}
```

Why `struct dog **`?

- `dList` is a pointer TO a pointer TO a struct dog
- `dList[i]` is a pointer to struct dog (needs its own malloc)
- This allows each record to be independently allocated and freed

Q4c — Read from binary file and store in dynamic list [3 marks]:

```
// Add dogData.bin to project (copy it to the project folder)
FILE *fp = fopen("dogData.bin", "rb");
if (!fp) {
    printf("Error opening file!\n");
    return -1;
}

struct dog temp;
int count = 0;

while (fread(&temp, sizeof(temp), 1, fp) == 1) {
    // Allocate memory for THIS record before storing
    dList[count] = (struct dog*)malloc(sizeof(struct dog));
    if (!dList[count]) {
        printf("Allocation failed for record %d\n", count);
        fclose(fp);
        return -1;
    }
    *dList[count] = temp;    // dereference to store the struct
    count++;
}
```

```
fclose(fp);
printf("Read %d records from file.\n", count);
```

Key point: You MUST call `malloc(sizeof(struct dog))` for each record before assigning to it. Without this, you're writing to a NULL pointer → crash.

Q4d — Display all records [1 mark]:

```
printf("\n--- Records from file ---\n");
for (int i = 0; i < count; i++) {
    if (dList[i] != NULL) {
        displayRecord(*dList[i]);    // dereference: pass struct not pointer
    }
}
```

Q4e — Delete the second record (index 1) and display again [3 marks]:

```
// Delete record at index 1 (the second record)
if (dList[1] != NULL) {
    free(dList[1]);
    dList[1] = NULL;    // MUST set to NULL after freeing
}

printf("\n--- After deleting record 2 ---\n");
for (int i = 0; i < count; i++) {
    if (dList[i] != NULL) {        // check for NULL before accessing
        displayRecord(*dList[i]);
    }
}
```

Always check `!= NULL` before accessing — after deleting, `dList[1]` is NULL and dereferencing it would crash.

Q4f — Free the collection COMPLETELY [1 mark]:

```
// Free each individual record first
for (int i = 0; i < capacity; i++) {
    if (dList[i] != NULL) {
        free(dList[i]);
        dList[i] = NULL;
    }
}

// Then free the list of pointers
free(dList);
dList = NULL;
```

Order matters: Free inner allocations first, then the outer list. If you `free(dList)` first, you lose all the pointers to the individual records — memory leak!

Q4g — Advantage/Disadvantage comment [5 marks]:

```
/*
 * ADVANTAGE of dynamic list of dynamic structs vs dynamic list of static structs:
 * Each record is allocated independently using its own malloc(). This means
 * individual records can be deleted cleanly by freeing just that pointer and
 * setting it to NULL, without affecting other records. In a dynamic list of
 * static structs, you cannot cleanly delete one record – you would need to
 * shift records or leave a gap.
 *
 * DISADVANTAGE of dynamic list of dynamic structs vs dynamic list of static structs:
 * Managing memory is significantly more complex. You must call malloc() twice –
 * once for the list of pointers and once for each individual record – and then
 * free() in the correct order. This increases the chance of memory leaks and
 * bugs such as dangling pointers or double-free errors. A dynamic list of static
 * structs only requires one malloc() for the entire list.
 */
```

Complete Question 4 Program:

```
#include "myHeader.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

// Copy function bodies from Q3 (or compile with myHeader.c separately)
struct dog fillRecord(void) {
    struct dog d;
    printf("Enter chip ID: "); scanf("%14s", d.chipId);
    printf("Enter name: ");    scanf("%29s", d.name);
    printf("Enter breed: ");   scanf("%29s", d.breed);
    return d;
}

void displayRecord(struct dog d) {
    printf("Chip: %s | Name: %s | Breed: %s\n", d.chipId, d.name, d.breed);
}

int main(void) {
    int capacity = 10;
    struct dog **dList = (struct dog**)malloc(capacity * sizeof(struct dog*));
    if (!dList) { printf("Allocation failed!\n"); return -1; }

    for (int i = 0; i < capacity; i++) dList[i] = NULL;

    FILE *fp = fopen("dogData.bin", "rb");
    if (!fp) { printf("Error opening file!\n"); return -1; }

    struct dog temp;
    int count = 0;
    while (fread(&temp, sizeof(temp), 1, fp) == 1) {
        dList[count] = (struct dog*)malloc(sizeof(struct dog));
        if (!dList[count]) { printf("Alloc failed!\n"); fclose(fp); return -1; }
    }
```

```

        *dList[count] = temp;
        count++;
    }
    fclose(fp);

    printf("--- All Records ---\n");
    for (int i = 0; i < count; i++) {
        if (dList[i]) displayRecord(*dList[i]);
    }

    free(dList[1]);
    dList[1] = NULL;

    printf("\n--- After Deleting Record 2 ---\n");
    for (int i = 0; i < count; i++) {
        if (dList[i]) displayRecord(*dList[i]);
    }

    /*
    * ADVANTAGE: Individual records can be deleted cleanly.
    * DISADVANTAGE: More complex memory management – must free each record AND the
    */

    for (int i = 0; i < capacity; i++) {
        if (dList[i]) { free(dList[i]); dList[i] = NULL; }
    }
    free(dList);
    dList = NULL;

    return 0;
}

```

Final Exam Submission Checklist

Before zipping and uploading, make sure you have:

- ☐ This Word document filled with all answers, screenshots, and source code
- ☐ Screenshots Q1a, Q1b, Q1c, Q1d (GCC commands + output)
- ☐ Screenshots Q2b (Visual Studio debugging session with breakpoints and Watch)
- ☐ Visual Studio solution with 3 projects: Question2, Question3, Question4
- ☐ myHeader.h header file in Question4 project
- ☐ dogData.bin binary file (generated by Question3, added to Question4)
- ☐ All projects compile without errors
- ☐ Zip folder named NameSurnameClass
- ☐ Upload to VLE using the correct link for your class